Lab 08: Control FSM

건양대학교 국방반도체공학과
백종섭 교수

Step 1: 설계 — control.sv

> `cpu_pkg.sv`는 lab02에서 이미 작성한 자산을 그대로 사용한다 (lab02-alu/cpu_pkg.sv 참조).

`control_blank.sv`를 열고 Comment #1 영역에 FSM을 작성한다.

```
// 1. opcode decode (조합)
state_t state;
logic is_op_memrd, is_not, is_brz, is_bra, is_sta, is_wfr;

assign is_op_memrd = (ir_opcode inside {ADD, AND, LDA});
assign is_not      = (ir_opcode == NOT);
assign is_brz      = (ir_opcode == BRZ);
assign is_bra      = (ir_opcode == BRA);
assign is_sta      = (ir_opcode == STA);
assign is_wfr      = (ir_opcode == WFR);

// 2. state FF
always_ff @(posedge clk or negedge rst_n)
   if (!rst_n)      state <= INST_ADDR;
   else if (!halt)  state <= state.next();
```

Step 1 (계속): 출력 FF


```
// 3. 출력 FF — next-state 기반
always_ff @(posedge clk or negedge rst_n) begin
   if (!rst_n) begin
      {mem_rd, ir_load, inc_pc, load_reg, load_pc, mem_wr} <= 6'b0;
      fetch_phase <= 1'b1;
      halt        <= 1'b0;
   end
   else if (!halt) begin
      fetch_phase <= (state.next() inside
         {INST_ADDR, INST_FETCH, INST_LOAD, INST_DECODE});
      case (state.next())
         INST_ADDR  : {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b000_000;
         INST_FETCH : {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b100_000;
         INST_LOAD  : {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b110_000;
         INST_DECODE: {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b110_000;
         OP_ADDR    : begin
            {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b001_000;
            halt <= is_wfr;
         end
         OP_FETCH   : begin
            mem_rd <= is_op_memrd;
            {ir_load, inc_pc, load_reg, load_pc, mem_wr} <= 5'b0;
         end
         OP_ALU     : begin
            mem_rd   <= is_op_memrd;  ir_load <= 1'b0;
            inc_pc   <= is_brz && zero;
            load_reg <= is_op_memrd | is_not;
            load_pc  <= is_bra;       mem_wr  <= 1'b0;
         end
         UPDATE     : begin
            {mem_rd, ir_load, inc_pc, load_reg, load_pc} <= 5'b0;
            mem_wr <= is_sta;
         end
         default    : {mem_rd,ir_load,inc_pc,load_reg,load_pc,mem_wr} <= 6'b0;
      endcase
   end
end
```

Step 2: TB — 전체 opcode 검증

`tb_control_blank.sv`를 열고 Comment #1, #2를 작성한다.

- Comment #1: drive_fsm task

```
// Comment #1 : drive_fsm task
task drive_fsm(input opcode_t op);
   ir_opcode = op;
   repeat(8) @(posedge clk);
endtask
```

- Comment #2: 전체 opcode 검증

```
// Comment #2 : 전체 opcode 검증
drive_fsm(ADD);
drive_fsm(BRA);
drive_fsm(STA);
drive_fsm(WFR);
```

💻 tb_control.sv

Step 3: 시뮬레이션

- 시뮬레이션하여 각 opcode의 제어 신호를 파형으로 확인한다.

```
cd sim
xrun -f lab08_blank.f -input ../../shm.tcl
```

Expected Waveform:

```
time  rst_n  state       ir_opcode  mem_rd  ir_load  inc_pc  load_reg  load_pc  mem_wr  halt
----  -----  ----------  ---------  ------  -------  ------  --------  -------  ------  ----
   0      0  INST_ADDR   WFR             0        0       0         0        0       0     0
 100      1  INST_ADDR   WFR             0        0       0         0        0       0     0
 200      1  INST_FETCH  ADD             1        0       0         0        0       0     0    #2
 300      1  INST_LOAD   ADD             1        1       0         0        0       0     0
 400      1  INST_DECODE ADD             1        1       0         0        0       0     0
 500      1  OP_ADDR     ADD             0        0       1         0        0       0     0
 600      1  OP_FETCH    ADD             1        0       0         0        0       0     0
 700      1  OP_ALU      ADD             1        0       0         1        0       0     0
 800      1  UPDATE      ADD             0        0       0         0        0       0     0
 900      1  INST_ADDR   ADD             0        0       0         0        0       0     0
1000      1  INST_ADDR   BRA             0        0       0         0        0       0     0
1100      1  INST_FETCH  BRA             1        0       0         0        0       0     0
1200      1  INST_LOAD   BRA             1        1       0         0        0       0     0
1300      1  INST_DECODE BRA             1        1       0         0        0       0     0
1400      1  OP_ADDR     BRA             0        0       1         0        0       0     0
1500      1  OP_FETCH    BRA             0        0       0         0        0       0     0
1600      1  OP_ALU      BRA             0        0       0         0        1       0     0
1700      1  UPDATE      BRA             0        0       0         0        0       0     0
1800      1  INST_ADDR   STA             0        0       0         0        0       0     0
1900      1  INST_FETCH  STA             1        0       0         0        0       0     0
2000      1  INST_LOAD   STA             1        1       0         0        0       0     0
2100      1  INST_DECODE STA             1        1       0         0        0       0     0
2200      1  OP_ADDR     STA             0        0       1         0        0       0     0
2300      1  OP_FETCH    STA             0        0       0         0        0       0     0
2400      1  OP_ALU      STA             0        0       0         0        0       0     0
2500      1  UPDATE      STA             0        0       0         0        0       1     0
2600      1  INST_ADDR   WFR             0        0       0         0        0       0     0
```

Step 4: 완성품 복사 + Git Checkin

검증 끝난 _blank.sv 파일을 lab00-design 폴더에 모듈명으로 복사하고 커밋한다.

```
cd ..
cp control_blank.sv ../lab00-design/control.sv

git status
git add control_blank.sv
git add ../lab00-design/control.sv
git commit -m "lab08: done"
git push
```